

Requirement

- The information which describes the user's expectation about the system performance is called as requirement.
- The requirement must be clear and unambiguous. Some requirement definition has to face problem of lack of clarity.

Characteristics of requirements

There are various characteristic of requirements. They are as follows :

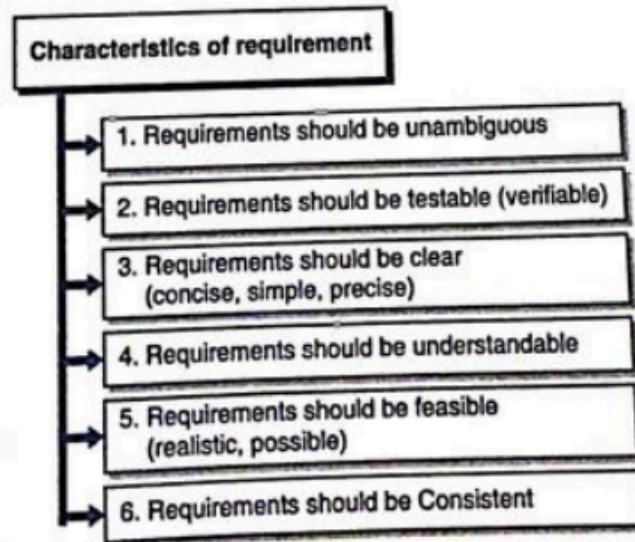


Fig. 2.1.1 : Characteristics of requirements

- Software Engineering (MU - Sem. 6 - Comp) 2-2 Requirements Analysis and Management
- **1. Requirements should be unambiguous**
 - Ambiguous means the single word or statement has more than one meaning. If requirements contain ambiguity then it is difficult to fulfill the requirements correctly.
 - Therefore the requirements should be unambiguous means non confusing.
 - **2. Requirements should be testable (verifiable)**
 - The requirements should be testable means tester should be able to easily test the requirements whether they are implemented successfully or not. For easy test, the requirement should be clear and unambiguous.
 - **3. Requirements should be clear (concise, simple, precise)**
 - The requirements should be clear. They should not contain any unnecessary information.
 - If there is any unnecessary information then it becomes difficult to achieve the appropriate fulfillment of the requirement.
 - **4. Requirements should be consistent**
- 2.1.1 A Q.1

- 4. **Requirements should be understandable**
- The requirements should be understandable means when anyone read it then it should be understood by that person. To make it understandable proper conventions are used.
 - It should be grammatically correct.
- 5. **Requirements should be feasible (realistic, possible)**
- The requirement should be completed within the given time and budget.
 - Specified requirement said to be feasible if it can be implemented using existing technology, estimated budget and time.
- 6. **Requirements should be Consistent**
- Consistency is an important feature of requirements. It means that all inputs must be processed similarly.
 - It should not happen that processes produce different outputs for same inputs coming from different sources.

Requirement Engineering

- The procedure which collects the software requirements from customer, analyze and document them is called as **requirement engineering**.
 - The aim of requirement engineering is to create and maintain 'System Requirements Specification' document.
 - Requirements engineering is the process of understanding and defining which services are required and identifying the constraints on these services.
 - Requirement engineering processes ensures your software will meet the user expectations, ending up with high quality software.
-
- It's a critical stage of the software process as errors at this stage will reflect later on in the next stages, which definitely will cause you higher costs.
 - At the end of this stage, a requirement document that specifies the requirements will be produced and validated with the stakeholders.

2.1.1 Activities Involved In Requirements Engineering

Q. 2.1.2 What are major tasks of requirement engineering? (Ref. sec. 2.1.1)

(5 Marks)

- The **activities/tasks involved in requirements engineering** vary widely, depending on the type of system being developed and the specific practices of the organization(s) involved.
- These may include :

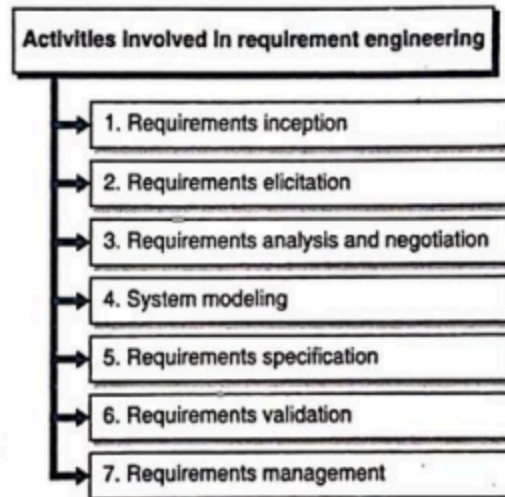


Fig. 2.1.2 : Activities involved in requirements engineering

→ 1. Requirements Inception

- Inception is a task where the requirement engineering asks a set of questions to establish a software process.
- In this task, it understands the problem and evaluates with the proper solution.
- It collaborates with the relationship between the customer and the developer.
- The developer and customer decide the overall scope and the nature of the question.

→ 2. Requirements Elicitation

- In requirements engineering, requirements elicitation is the practice of researching and discovering the requirements of a system from users, customers, and other stakeholders. The practice is also sometimes referred to as "requirement gathering".
- We will see more details regarding requirement gathering in section 2.2.

→ 3. Requirements Analysis and Negotiation

- Requirements are identified (including new ones if the development is iterative) and conflicts with stakeholders are solved. Both written and graphical tools (the latter commonly used in the design phase but some find them helpful at this stage, too) are successfully used as aids.
- Examples of written analysis tools : use cases and user stories.
- Examples of graphical tools : UML and LML.

→ 4. System modelling

- Some engineering fields (or specific situations) require the product to be completely designed and modeled before its construction or fabrication starts and, therefore, the design phase must be performed in advance.
- For instance, blueprints for a building must be elaborated before any contract can be approved and signed.
- Many fields might derive models of the system with the Lifecycle Modeling Language, whereas others might use UML.

Note : In many fields, such as software engineering, most modeling activities are classified as design activities and not as requirement engineering activities.

→ 5. Requirements specification

- Requirements are documented in a formal artifact called Requirements Specification (RS).
- Nevertheless, it will become official only after validation. A RS can contain both written and graphical (models) information if necessary.
- Example : Software Requirements Specification (SRS).

→ 6. Requirements validation

It is the process of checking whether the documented requirements and models are consistent and meet the needs of the stakeholder. Only if the final draft passes the validation process, the RS becomes official.

→ 7. Requirements management

Managing all the activities related to the requirements since inception, supervising as the system is developed and, even until after it is put into use (e. g., changes, extensions, etc.)

Requirement Elicitation

- In requirements engineering, requirements elicitation is the practice of researching and discovering the requirements of a system from users, customers, and other stakeholders. The practice is also sometimes referred to as "requirement gathering".
- The term elicitation is used in books and research to raise the fact that good requirements cannot just be collected from the customer, as would be indicated by the name requirements gathering.
- Requirements elicitation is non-trivial because you can never be sure you get all requirements from the user and customer by just asking them what the system should do OR NOT do (for Safety and Reliability).
- Requirements elicitation practices include interviews, questionnaires, user observation, workshops, brainstorming, use cases, role playing and prototyping.
- Before requirements can be analyzed, modeled, or specified they must be gathered through an elicitation process. Requirements elicitation is a part of the requirements engineering process, usually followed by analysis and specification of the requirements.

There are different ways to identify customer requirement :

(1) Interviews

- Interviews are important way for gathering requirements. Organization may take various kinds of interviews such as :
 - o Structured or closed interviews in which information to be collected from customer is decided in advance.
 - o Non-structured or open interviews in which details needs to be collected from customer are decided in advance.
 - o Oral interviews.
 - o Written interviews.
 - o Face-to-face interviews which are held among two people across the table.
 - o Groups of persons are participating in group interviews. They support to cover any missing requirement as number of people participates in this process.

(2) Surveys

- Organization may perform surveys of different stakeholders by questioning them about their requirements and expectation from the proposed software system.
- The advantage of this technique is that, collecting requirements is economically beneficial because it collects requirements from a large number of persons at same time.
- Surveys are less effective method of data discovery.
- Frequently, a survey's main finding is that other questions should have been asked instead. Surveys are most useful for capturing clear factual information.



(3) Questionnaires

- Questionnaires are a document way which contains in-built set of objectives that is based on questions and their options.
- This document is given to all stakeholders to give answer of those questions which are gathered and compiled.
- Output of this mechanism is, if an answer for some question is not given in the questionnaire, the issue may be remaining unattended.
- The advantage of this technique is that, collecting requirements economically is benefited because it collects requirements from huge amount of persons at same time.
- Questionnaires are less effective method of data discovery.

(4) Task analysis

- In this technique, team of software developers and engineers may identify the functional specifications for which the new system is needed to develop.
- If the customer has some software to do the particular operations then it is analyzed by this technique to find out requirements for proposed system.

(5) Domain Analysis

- Each software put into some domain category.
- The experienced persons in that domain can be a great support to study general and specific requirements.

(6) Brainstorming

Brainstorming is an informal debate held between different stakeholders and all their suggestions are documented for further requirement analysis.

(7) Prototyping

- Prototyping is a technique in which, we create user interface without including the functionality for user to interpret features of desired software product.
- It supports providing details about requirements.
- If the client does not know its own requirements, in such case the developer develop a prototype based on requirements provided at initial stage.
- The prototype is displayed to the client and the feedback is collected from client.
- The client feedback is used as an input for requirement gathering.

(8) Observation

- Team of experienced persons visits the organization or workplace of client.
- They observe existing system's work.
- They observe the flow of control at client's end and how execution problems are dealt. The team itself draws some conclusions which aid to form requirements expected from the software.

2.3 Requirement Analysis

Q. 2.3.1 Write note on Requirements analysis. (Ref. sec. 2.3)

(5 Marks)

- Requirements analysis is vital phase of requirement engineering process after Elicitation.
- In this activity, we understand and refine the collected requirements to make them consistent and non-confusing.
- This activity reviews each and every requirement and may also give graphical view of the overall system.
- After the analysis, it is supposed that the understanding of requirements of project are increased significantly.
- In this activity, developer can communicate with customer to clear confusing points and to understand which requirements are more vital than others.
- The significant factors in the requirement analysis activity are :
 1. Identify and solve confusion among requirements at the same level and among different levels.
 2. Identify the boundaries of the proposed software system and the way in which it communicate with environment.
 3. Evaluate customer requirements for overall system requirements and then break down them into individual component level.
- The objective of the Requirement Analysis is generating a solution to implement the requirements.
- Process of the Requirements Analysis includes following steps :
 1. Analysis of the requirements
 2. Description of the solution
 3. Cost estimate and prioritization.
- Here are some examples of how we can consider problems occurred in requirement analysis.
- Stakeholders don't recognize what they really required.
- Stakeholders state the requirements in their own words.
- Various stakeholders may have different and confusing requirements.
- Organizational and political factors may affect the software needs.
- The requirements are modified at the time of requirement analysis process.
- New stockholders may be included and the business environment may change.

- The requirements, which are in general assumed, are divided into three categories, functional requirements, non-functional requirements, and domain requirements.

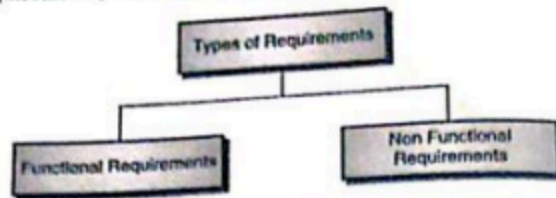


Fig. 2.4.1 : Types of requirements

2.4.1 Functional Requirements

Q. 2.4.2 Explain Functional Requirements. (Ref. sec. 2.4.1)

- The Functional requirements specification create document regarding the operations as activities which a system must be able to carry out.
- In software engineering, a functional requirement is used to define a function regarding a system or its component, in which a function is described as a specification of behavior between inputs and outputs.
- In the functional requirements there may be elements such as calculations, technical details, manipulation and processing, and other specific functionality which define what a system is supposed to implement.
- As defined in requirements engineering, functional requirements is used to mention the results of a system.
- This should be contrasted with non-functional requirements that mention overall characteristics like as cost and reliability.
- Functional requirements drive the application architecture of a system, while non-functional requirements drive the technical architecture of a system.
- **Functional Requirements should include :**
 - o Descriptions of data to be entered into the system.
 - o Descriptions of operations performed by each screen.
 - o Descriptions of work-flows performed by the system.
 - o Descriptions of system reports or other outputs.
 - o Persons who can enter the data into the system.
 - o Way the system meets applicable regulatory requirements.

2.4

Examples of Functional Requirements

Functional requirements must contain functions performed by specific screens, outlines of work performed by the system, and other business or compliance requirements the system must meet.

☛ **Interface requirements**

- Field 1 accepts numeric data entry.
- Field 2 only accepts dates before the current date.
- Screen 1 can print on-screen data to the printer.

☛ **Business Requirements**

- Data must be entered before a request can be approved.
- Clicking the Approve button moves the request to the Approval Workflow.
- All personnel using the system will be trained.

☛ **Regulatory/Compliance Requirements**

- The database will have a functional audit trail.
- The system will limit access to authorized users.
- The spreadsheet can secure data with electronic signatures.

☛ **Security Requirements**

- Members of the Data Entry group can enter requests but cannot approve or delete requests.
- Members of the Managers group can enter or approve a request but cannot delete requests.
- Members of the Administrators group cannot enter or approve requests but can delete requests.

2.4.2 Non Functional Requirements (NFR)

➔ (MU - Dec. 16)

Q. 2.4.3 Explain Non-Functional Requirements with types. (Ref. sec. 2.4.2)

Dec. 16, 4 Marks

- These are basically the quality constraints that the system must satisfy according to the project contract.
- The priority or extent to which these factors are implemented varies from one project to other. They are also called non-behavioral requirements. They basically deal with issues like:
 - o Portability
 - o Security
 - o Maintainability
 - o Reliability
 - o Scalability
 - o Performance
 - o Reusability
 - o Flexibility
- NFRs are classified into following types :
 - o Interface constraints
 - o Performance constraints : response time, security, storage space, etc.

- Operating constraints
- Life cycle constraints : maintainability, portability, etc.
- Economic constraints

- The process of specifying non-functional requirements needs the knowledge of the functionality of the system, and also the knowledge of the context within which the system will operate.

2.4.2(A) Types of Non-Functional Requirements

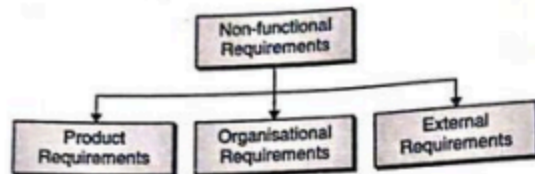


Fig. 2.4.2 : Types of non-functional requirements

- **Product requirements** : Requirements which mention that the delivered product should have a specific behavior in a specific way, e.g. execution speed, reliability etc.
- **Organizational requirements** : Requirements that are a result of organizational policies as well as procedures, such as process standards which has been used, implementation necessities etc.
- **External requirements** : Requirements which are generated from the factors that are external to the system and its development process, e.g. interoperability requirements, legislative requirement.

2.4.3 Domain Requirements

- Domain requirements are used to describe system characteristics as well as features which impact the domain. Those may be new functional requirements, constraints on present requirements or may define particular computations.
- If domain requirements are not satisfied, the system may be unworkable.
- Example : Library system

Syllabus Topic : Software Requirement Specification (SRS)

2.5 Software Requirement Specification (SRS)

→ (MU - May 15)

Q. 2.5.1 What is SRS document ? (Ref. sec. 2.5)

May 15, 4 Marks

- SRS stands for Software/System Requirement Specification. SRS is a special kind of document which contains user requirements for a system which states properties and constraints that must be satisfied by a software system.
- IEEE defines software requirements specification as, 'a document that clearly and precisely describes each of the essential requirements (functions, performance, design constraints and quality attributes) of the software and the external interfaces.'

- The outcome of the requirements gathering and analysis phase of the SDLC is **Software Requirements Specification (SRS)**.
- SRS is also called as **requirements document**.
- SRS is base for software engineering actions and is generated when all requirements of software project are gathered and analyzed.
- It is generally signed at the end of requirements engineering phase.
- Following are 6 requirements stated by Heninger, which SRS document should follow :
 1. SRS document should specify external system behaviour.
 2. SRS document should specify implementation constraints.
 3. It should be easily changeable if any changes occur.
 4. It should act as reference tool for maintaining the system.
 5. SRS document record forethought about the lifecycle of the system i.e. predicts changes.
 6. It must include acceptable response to undesired events.

2.5.1 Advantages of SRS

Q. 2.5.2 What are advantages of SRS? (Ref. sec. 2.5.1) (4 Marks)

- (1) SRS contains the base for agreement among the client and the company who develop the product as per expectations.
- (2) A SRS gives a base for verification of the completely developed product.
- (3) Using high-quality SRS, we can develop high-quality software product.
- (4) A high-quality SRS decreases the cost of development.

2.5.2 Characteristics of SRS

Q. 2.5.3 Explain any five characteristics of SRS. (Ref. sec. 2.5.2) (10 Marks)

A good software requirement specification must satisfy following characteristics.

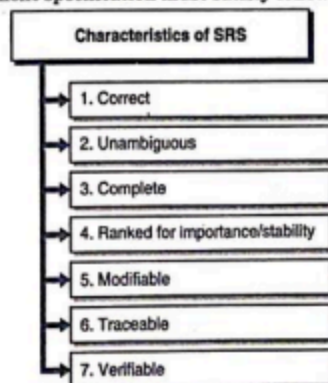


Fig. 2.5.1 : Characteristics of SRS



- **1. Correct**
 - SRS is correct when all requirements of user are described in the requirement document.
 - The listed requirements must be matched with desired system.
 - This depict that every requirement is analyzed to give assurance that it (SRS) contains user requirements.
 - Remember that there is no specific tool or process to ensure the correctness of SRS.
 - Correctness gives assurance that all stated requirements are worked as expected.
- **2. Unambiguous**
 - SRS does not contain any confusion when each specified requirement has single interpretation.
 - This characteristic state that every requirement is individually interpreted.
 - In situation, where one term has number of meanings, then its meaning must be specified in the SRS so that it will be non-confusing and simple to understand.
- **3. Complete**
 - SRS is complete when the requirements undoubtedly specified that which work the software product is required to perform.
 - This contains each and every requirement associated to performance, design and functionality.
- **4. Ranked for importance/stability**
 - Each and every requirement has not same importance, so every requirement is recognized to make differentiation between requirements.
 - For this purpose, it is needed to undoubtedly recognize every requirement.
 - Stability refers to the probability of further modifications in the requirement.
- **5. Modifiable**

The requirements given by user are changeable, so requirement document must be generated in such a way that those modifications can be included easily in SRS by preserving consistently the structure and style of the SRS.
- **6. Traceable**
 - SRS is observable when the source of every requirement is unambiguous and facilitates the description of every requirement in future.
 - Forward tracing and backward tracing methods are used for this purpose.
 - Forward tracing state that every requirement must be referencing to design and code of components.
 - Backward tracing state every requirement explicitly addressing its source.
- **7. Verifiable**
 - SRS is testable when the stated requirements can be tested with a cost-effective procedure to verify whether the final software fulfills those requirements.

- The requirements are tested through reviews.
- Remember that clear requirement is needed for verifiability.

2.5.3 Format of SRS

Q. 2.5.4 Explain general format of Software Requirement Specification (SRS). (Ref. sec. 2.5.3) **(10 Marks)**

Structure of SRS document includes following sections with various subsections in it :

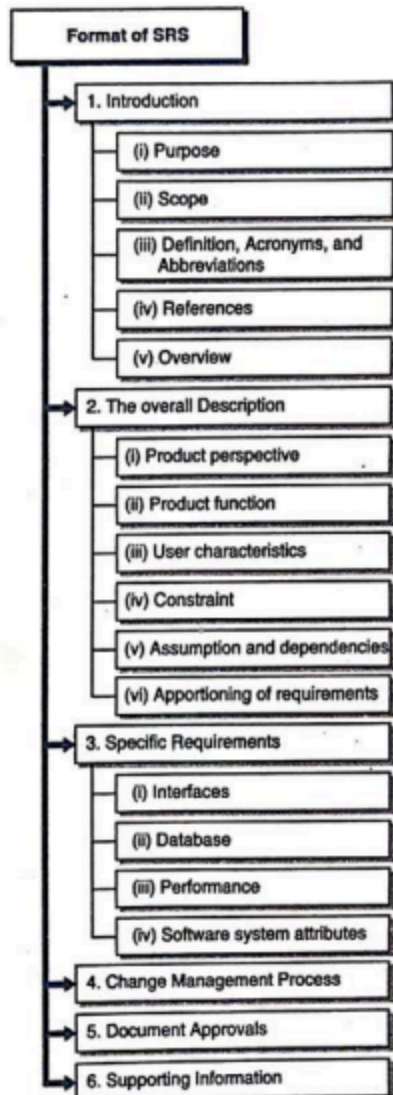


Fig. 2.5.2 : Format of SRS



1. Introduction

Introduction contains brief information about the SRS. This part is further divided into following subsections :

(i) Purpose

This section specifies the main purpose of SRS document with its intended audience for whom SRS is constructed.

(ii) Scope

- This section specifies scope of the system for which SRS is built.
- Scope defines not only benefits, objectives and behaviour of software product but also limitations of software so as to understand the boundary of software.

(iii) Definition, Acronyms, and Abbreviations

To avoid ambiguity of terms specified in requirements, SRS provides definition, abbreviation and acronyms about these terms.

(iv) References

It contains list of all documents which are referred by this document.

(v) Overview

It gives overview of the document including its goals and objectives of the system.



2. The overall Description

It provides overall information about the requirements. Customers/users of the system are concerned about this section. It contains following subsections :

(i) Product perspective

This section contains the information which states the benefits of current product over other existing products.

(ii) Product function

- The functionality of software product summarizes in this section of SRS document.
- It is possible to use diagrams to summarize the software functionality and logical relationship among variables present in the system. This information is provided in simple way so that users of the system can understand.

(iii) User characteristics

To use any system, it is mandatory that users of system should have some basic knowledge i.e. at least the educational qualification or basic knowledge related to the field. This section provides the criteria about the users of the system.

(iv) Constraint

It includes limitations of components used in the system for example, hardware limitations.

(v) Assumption and dependencies

It contains the list of factors on which SRS is depending. That is if the factor changes it leads to change in the SRS too.

(vi) Apportioning of requirements

It states the order in which the specified requirements are fulfilled.

→ **3. Specific Requirements**

This section of SRS actually specifies the Requirements briefly by taking help of information stated in above section. This section is divided into following subsections :

(i) Interfaces

- This section of SRS contain the information about various interfaces that are needed for product, like hardware/software interface, system interface, user interfaces, and communication interfaces.
- It means that each and every input device, output device and the communication medium used by system is introduced in this section of SRS document.

(ii) Database

To store data required and generated by system, database is required. Information about the database used in the system is given in this section.

(iii) Performance

It describes the system performance which includes the required speed, task completion time, response time and throughput of the system.

(iv) Software system attributes

- System attributes are : reliability, security, maintainability.
- This section of SRS provides the assurance that all system attributes will be provided by product and the way in which terms these attributes are satisfied/ fulfilled.

→ **4. Change Management Process**

- Due to additional user requirements, changes in the system are occurred. This section provides a way that will help to handle such kind of changes.
- As the system make any changes according to user requirements, SRS document should be changed accordingly.

→ **5. Document Approvals**

- The SRS document should be accepted by both the parties, including the customers for whom the system is developing and the developer who develop the system.

- Document Approvals section include the approval date time and signature
- 6. Supporting Information
 - It includes guidelines about how to use SRS, index of SRS, references, etc.

2.5.4 SRS Document for Online Student Feedback System

→ (MU - May)

Q. 2.5.5 Build an SRS document for online student feedback system.
(Ref. sec. 2.5.4)

May 15, 9 March

Table of Contents

1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Overview
2. General Description
 - 2.1 User Manual
3. Functional Requirements
 - 3.1 Description
4. Interface Requirements
 - 4.1 GUI
 - 4.2 Hardware Interface
 - 4.3 Software Interface
5. Performance Requirements
6. Design Constraints
7. Other non-functional Attributes
 - 7.1 Security
 - 7.2 Reliability
 - 7.3 Availability
 - 7.4 Maintainability
 - 7.5 Reusability
8. Operational Scenarios
9. Preliminary Schedule

1. Introduction

1.1 Purpose

This document gives detailed functional and non-functional requirements for online student feedback system. The purpose of this document is that the requirements mentioned in it should be utilized by software developers to implement the system.

1.2 Scope

This system allows the students to provide quick feedback which is provided by collage staff. The feedback report is generated and which is checked by HOD's. He can view grade and grade obtained to the lecturers.

1.3 Overview

This system provides an easy solution to collage staff and students for maintaining feedback related to collage staff and infrastructure, facility.

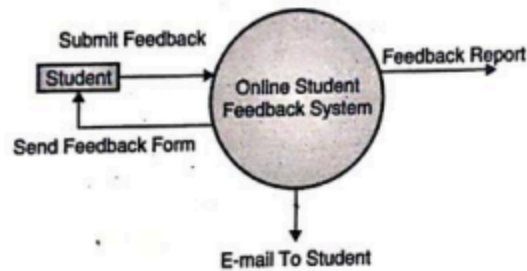


Fig. 2.5.3 : Online student feedback system

2. General Description

- This online student feedback system replaces the traditional, manual feedback system by which lots of paper work will be reduced. The teachers are able to provide feedback regarding facility and students are able to provide feedback easily. This is primary feature of this system.
- Another feature is that feedback form can be provided to student and staff by emailing for filling it.

2.1 User Manual

The system should provide Help option in which how to operate system should be explained. Also hard copy of this document should be given to user in booklet form.

3. Functional Requirement

3.1 Description

- For identity of staff, system should display staff photograph along with their names for that corresponding subject and skills.
- Statistical report accordingly subject or skill should display individual's report whenever required.

7.3 Availability

The system should be available during college hours.

7.4 Maintainability

There should be facility to add and delete feedback form for different purpose.

7.5 Reusability

The same system will be used in each semester.

8. Operational Scenarios

- There should be student database and teacher database. The student database should contain name and feedback information.
- The teacher database should contain name, subject, skills, and other details.

9. Preliminary Schedule

The system should be designed within 6 months.